

SISTEMAS OPERATIVOS

Conferencia 5 · Tema III: Gestión de Entrada/Salida

Carrera: Ingeniería Informática	Profesora: Ing. Claudia Vecino Fernández
Año: 3ro Período: 2do Tipo: CRD	Correo: claudiavf@uho.edu.cu
Disciplina: Infraestructura de Sistemas Informáticos	Teléfono: 54196249

Objetivo: Analizar la arquitectura de gestión de Entrada/Salida en los sistemas operativos: clasificación de dispositivos, controladores, DMA, capas de software de E/S, planificación de discos y tecnologías modernas de almacenamiento.

Sumario

- Dispositivos de E/S: clasificación y controladores.
- Acceso Directo a Memoria (DMA). Intercalación de sectores.
- Software de E/S: metas y arquitectura en capas.
- Manejadores de interrupción (Interrupt Handlers).
- Manejadores de dispositivos (Device Drivers).
- Software independiente del dispositivo.
- Software de E/S a nivel de usuario. SPOOLING.
- Ejemplos de manejadores: reloj y disco RAM.
- Discos magnéticos: características físicas y tiempos de acceso.
- Algoritmos de planificación de disco: FCFS, SSTF, SCAN, C-SCAN, LOOK, C-LOOK.
- Tecnologías modernas: SSD, NVMe, RAID, almacenamiento en nube.

Bibliografía

- Tanenbaum, A. S. Sistemas Operativos Modernos, 4ª ed. Págs. 233–339.
- Silberschatz, Galvin, Gagne. Operating System Concepts, 10ª ed. Caps. 12–13.
- Tanenbaum, A. S. Operating System: Design and Implementation. Págs. 110–186.
- Love, R. Linux Kernel Development, 3ª ed. Cap. 13–14 (Block I/O Layer).

1. Dispositivos de Entrada/Salida y Controladores

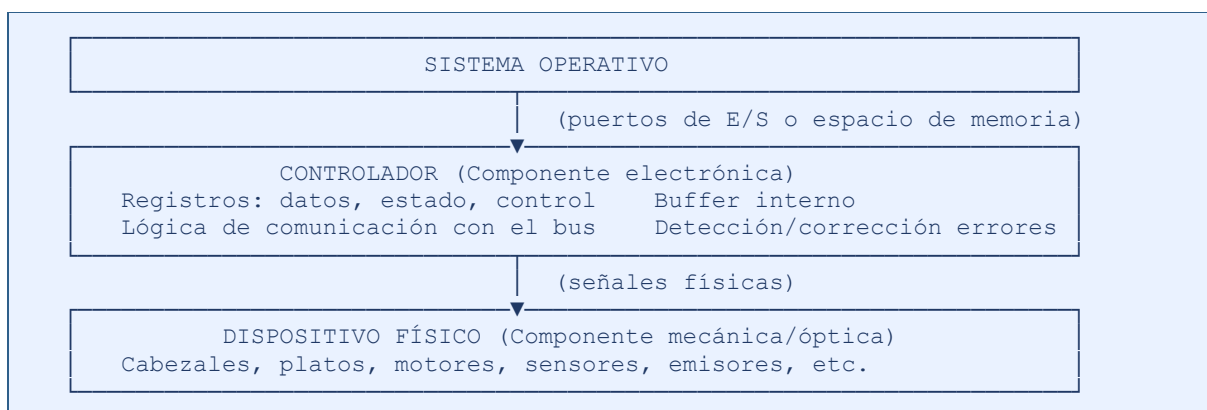
Una de las funciones principales de todo sistema operativo es la administración de los recursos, y dentro de estos los dispositivos de E/S ocupan un lugar central. El SO envía órdenes a los dispositivos, captura las interrupciones de fin de operación y gestiona errores, brindando además una interfaz simple y uniforme.

1.1 Clasificación de Dispositivos

Tipo		Características	Ejemplos
Dispositivos de bloque	de	Almacenan información en bloques de longitud fija direccionables independientemente. Permiten acceso aleatorio (lectura/escritura de un bloque sin afectar a otros).	Discos magnéticos (HDD), SSDs, cintas magnéticas, memorias flash USB.
Dispositivos de carácter	de	Entregan o reciben una cadena de caracteres. No tienen estructura de bloques, no son direccionables ni permiten búsqueda.	Teclados, ratones, interfaces de red, impresoras, terminales.
Dispositivos especiales		No encajan en los grupos anteriores: generan eventos periódicos, no transfieren datos.	Relojes de tiempo real (RTC), temporizadores de intervalo programable (PIT).

1.2 Estructura de un Dispositivo: Componente Mecánica y Controlador

Los dispositivos de E/S constan de dos partes: la componente mecánica (el dispositivo físico en sí) y la componente electrónica, denominada controlador (controller) o adaptador (adapter). El SO interactúa exclusivamente con el controlador, no con el dispositivo directamente.



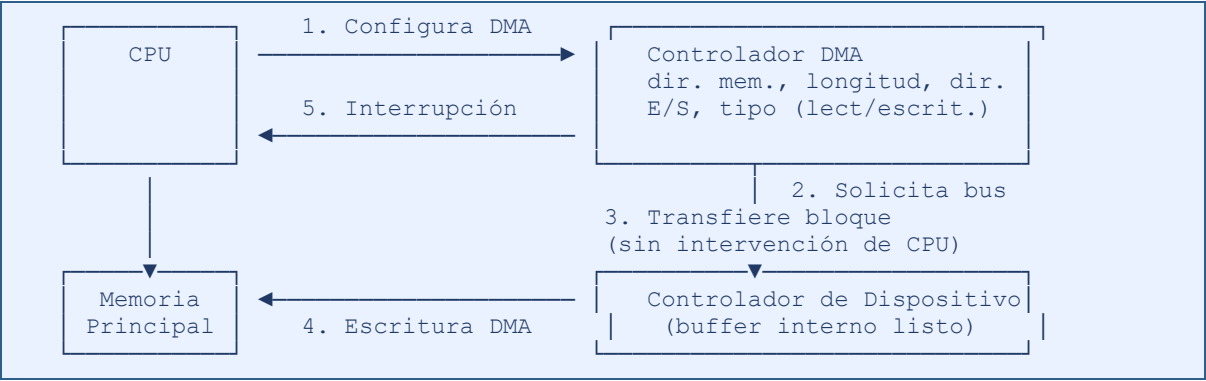
El controlador dispone de registros de comandos, estado y datos para comunicarse con la CPU. En algunas arquitecturas estos registros forman parte del espacio de memoria normal (memory-mapped I/O); en otras existe un espacio de direcciones separado (port-mapped I/O, instrucciones IN/OUT en x86). El SO escribe comandos en los registros, el controlador ejecuta la operación y emite una interrupción al finalizar.

✚ **Nota actualizada:** En arquitecturas modernas (x86-64, ARM) predomina el modelo memory-mapped I/O, donde los registros del controlador aparecen como direcciones de memoria normales. Los buses actuales son PCIe (Peripheral Component Interconnect Express) para discos NVMe y GPUs, USB 3.x/4.0 para periféricos, y SATA para discos tradicionales. El descubrimiento y configuración de dispositivos se realiza automáticamente mediante Plug & Play (PnP) y el estándar ACPI.

2. Acceso Directo a Memoria (DMA)

Sin DMA, la CPU debe transferir cada byte (o palabra) entre el buffer del controlador y la memoria principal, lo que consume tiempo de procesador en una tarea de bajo nivel. El DMA libera a la CPU de esta responsabilidad.

2.1 Funcionamiento del DMA



Proceso: (1) La CPU programa el controlador DMA con dirección de memoria destino, cantidad de bytes y dirección del dispositivo. (2-4) El DMA obtiene el control del bus y transfiere el bloque directamente entre el controlador del dispositivo y la memoria, sin intervención de la CPU. (5) Al terminar, el DMA genera una interrupción. La CPU solo gestiona el inicio y el fin de la transferencia.

Aspecto	E/S programada (sin DMA)	Con DMA
Intervención CPU	Copia byte a byte (muy intensiva)	Solo configura y recibe la interrupción final
Velocidad	Lenta; CPU bloqueada durante la transferencia	Rápida; CPU libre para otras tareas
Uso típico	Dispositivos de baja velocidad (teclado, ratón)	Discos, tarjetas de red, USB 3.x, NVMe
Coherencia de caché	No hay problema (CPU controla todo)	Requiere flush/invalidación de caché (cache coherency)

2.2 Intercalación de Sectores (Interleaving)

Cuando el controlador transfiere un sector leído al DMA/memoria, el disco sigue girando. Si se necesita el siguiente sector consecutivo, ya habrá pasado bajo la cabeza y habrá que esperar una rotación completa (~5–10 ms). La solución es la intercalación: numerar los sectores físicos con saltos, de modo que cuando el controlador termina de procesar el sector N, el sector N+1 justo llega bajo la cabeza.

Sin intercalación:	0	1	2	3	4	5	6	7	8	9	10	11
Intercalación 1:	0	6	1	7	2	8	3	9	4	10	5	11
Intercalación 2:	0	4	8	1	5	9	2	6	10	3	7	11
Leer sectores 0→1→2 sin interleave: 0 (leer) → espera 1 rot. → 1 (leer)...												
Leer sectores 0→1→2 con interleave: 0 (leer+transferir) → 1 ya está listo.												

📌 **Nota actualizada:** En los discos HDD modernos la intercalación la gestiona automáticamente el firmware del controlador (no el SO). Los SSDs y NVMe no tienen este problema porque carecen de partes móviles: el acceso es uniforme e instantáneo (~0,1

ms). El factor de intercalación óptimo depende de la velocidad de transferencia del bus y la latencia del controlador.

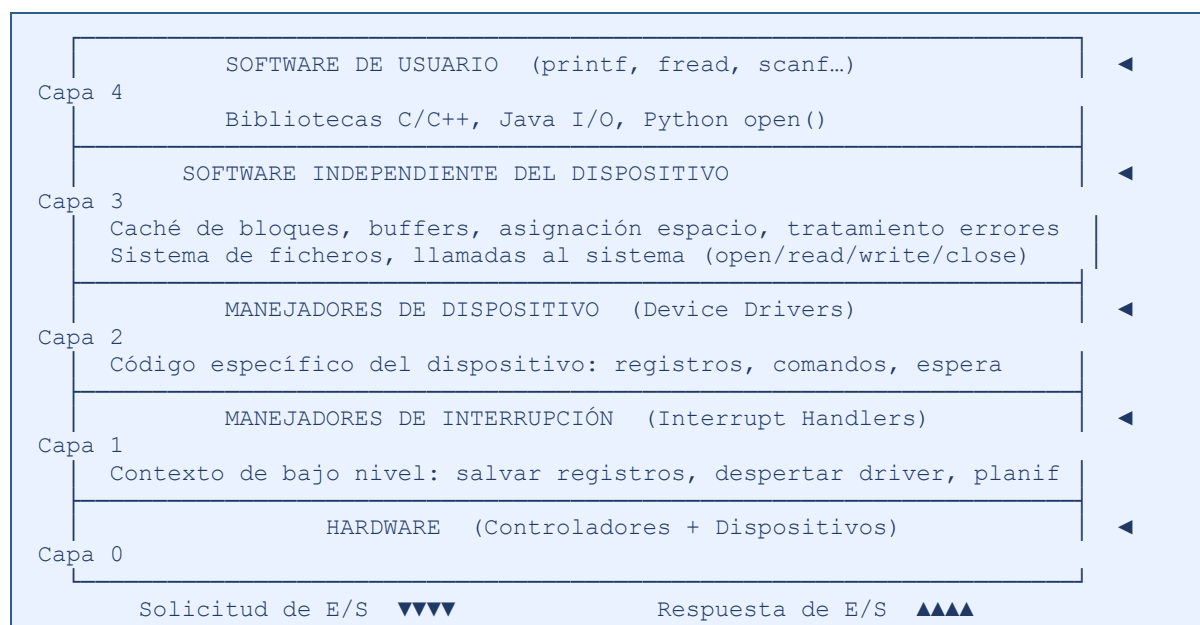
3. Software de Entrada/Salida: Metas y Arquitectura en Capas

El software de E/S se organiza en capas donde las más bajas ocultan las peculiaridades del hardware y las más altas presentan una interfaz uniforme al usuario. Cada capa solo interactúa con sus capas adyacentes, lo que facilita la portabilidad y el mantenimiento.

3.1 Metas del Software de E/S

Meta	Descripción
Independencia del dispositivo	Un programa puede usar cualquier dispositivo sin modificarse ni recompilarse. Ej.: <code>sort <input> >output</code> funciona igual si <code>input</code> es fichero, teclado o red.
Nominación uniforme	Los nombres de dispositivos y ficheros siguen el mismo convenio. En UNIX, todo es un fichero (ficheros, directorios, dispositivos bajo <code>/dev</code>).
Tratamiento de errores	Los errores se manejan lo más cerca posible del hardware. El controlador intenta recuperarse; si no, el device driver; solo si falla, se notifica a capas superiores.
Imagen sincrónica	Las E/S son asíncronas (la CPU inicia y sigue trabajando) pero el SO presenta una vista sincrónica a los procesos de usuario (el proceso se bloquea hasta que termina).
Garantía de operación	El SO garantiza acceso exclusivo a dispositivos dedicados (impresoras) y acceso ordenado a dispositivos compartidos (discos), evitando conflictos.

3.2 Arquitectura en Capas del Sistema de E/S



4. Las Cuatro Capas del Software de E/S

4.1 Manejadores de Interrupción (Interrupt Handlers)

Cuando un controlador finaliza una transferencia, emite una interrupción hardware. Por hardware se guarda en la pila la dirección de retorno y se consulta el vector de interrupciones para obtener la dirección del handler correspondiente, que comienza a ejecutarse automáticamente.

Secuencia de operaciones del manejador de interrupción:

#	Acción del Manejador de Interrupción
1	Salvar los registros del proceso interrumpido (en la tabla de procesos o en el stack del kernel).
2	Fijar el contexto del manejador del dispositivo (stack, segmentos de memoria).
3	Avisar al manejador del dispositivo que la transferencia finalizó (bandera, semáforo V, mensaje).
4	Invocar al planificador de procesos (si existe derecho de prioridad, puede haber cambio de contexto).
5	Restaurar el proceso seleccionado por el planificador o el proceso anterior si no hubo cambio.



Concepto clave: En sistemas modernos se distingue la Top Half (parte superior) de la Bottom Half de una interrupción. La Top Half es el handler inmediato (mínimo código, alta prioridad): guarda registros y encola trabajo. La Bottom Half (tasklets, work queues en Linux) realiza el trabajo pesado diferido, con las interrupciones habilitadas, mejorando la latencia del sistema.

4.2 Manejadores de Dispositivos (Device Drivers)

Todo el código de E/S específico del dispositivo reside en el device driver. Cada driver atiende un tipo de dispositivo (o familia similar). Recibe solicitudes abstractas de las capas superiores y las traduce en comandos concretos al controlador.

Flujo de operación del device driver:

#	Acción del Device Driver
1	Recibir solicitud de capa superior (operación, dirección, cantidad). Si el dispositivo está ocupado → encolar.
2	Decidir las operaciones necesarias y su secuencia (ej.: para disco: seek, rotational wait, transfer).
3	Escribir comandos e información en los registros del controlador.
4	Esperar la interrupción del controlador (bloqueo en multiprogramación; espera ocupada en monoprogramación).

Sistemas Operativos Conferencia 5		Ingeniería Informática · 3er Año
#	Acción del Device Driver	
5	Verificar el resultado: si hay error, reintentar. Si el error persiste, notificar a la capa superior.	
6	Transferir datos/estado a la capa superior y atender la siguiente solicitud de la cola.	

Cada dispositivo tiene una cola de solicitudes pendientes (similar a la cola de listos del planificador de CPU). Existe un planificador por cada cola de dispositivo, cuya complejidad varía según el tipo de dispositivo.

🔴 **Nota actualizada:** En Linux, los device drivers se implementan como módulos del kernel (.ko) cargables dinámicamente (modprobe/insmod). El framework más moderno para dispositivos es el Device Tree + DM (Device Mapper) para almacenamiento, libusb para USB, y V4L2 para vídeo. En Windows, los drivers usan el modelo WDM (Windows Driver Model) o el más reciente KMDF/UMDF.

4.3 Software Independiente del Dispositivo

Esta capa implementa funciones comunes a todos los dispositivos, abstraídas de sus peculiaridades. Actúa como intermediario entre los drivers y el software de usuario.

Función	Descripción
Interfaz uniforme	Presenta una interfaz homogénea (open, read, write, close, ioctl) independiente del dispositivo.
Conversión de nombres	Traduce el nombre simbólico del dispositivo (/dev/sda1) a la dirección del driver correspondiente.
Control de acceso	Verifica permisos: solo usuarios/procesos autorizados pueden acceder al dispositivo.
Tamaño de bloque uniforme	Abstrae diferencias en tamaño de sector físico; presenta bloques lógicos de tamaño uniforme.
Buffering y caché	Gestiona buffers para alinear las transferencias al tamaño de bloque del dispositivo; implementa page cache para discos.
Asignación de espacio	Gestiona la asignación de espacio en disco (bloques libres) y en dispositivos dedicados.
Tratamiento de errores	Recibe errores persistentes de los drivers y decide cómo notificarlos (señal, excepción, código de error).

4.4 Software de E/S a Nivel de Usuario: Bibliotecas y SPOOLING

Aunque la mayor parte del software de E/S reside en el kernel, una pequeña porción se enlaza con los programas de usuario en forma de bibliotecas. Estas funciones (printf, scanf, fopen, etc.) formatean los datos y realizan las llamadas al sistema correspondientes.

```
/* Ejemplo: printf() → write() en la cadena de llamadas */
printf("Hola %s\n", nombre);
└─ sprintf() formatea la cadena en buffer
    └─ fwrite() copia al buffer de la biblioteca stdio
        └─ write(fd, buffer, n) ← llamada al sistema
            └─ sys_write() en el kernel
```

dispositivo $\rightarrow$ vfs_write() $\rightarrow$ driver $\rightarrow$ controlador $\rightarrow$

SPOOLING (Simultaneous Peripheral Operations On-Line): técnica que permite a múltiples procesos compartir dispositivos dedicados (como impresoras) sin bloquearse mutuamente. Un proceso demonio (daemon) es el único autorizado a usar el dispositivo; los demás depositan sus trabajos en una cola (spool directory).

```
Proceso A  $\rightarrow$  /var/spool/lp/jobA
Proceso B  $\rightarrow$  /var/spool/lp/jobB
Impresora
Proceso C  $\rightarrow$  /var/spool/lp/jobC
```

El demonio es el ÚNICO proceso que accede directamente a la impresora.
Los demás procesos nunca bloquean la impresora; depositan y continúan.

✦ **Nota actualizada:** En Linux moderno, CUPS (Common UNIX Printing System) implementa el SPOOLING de impresión. El demonio de red (NetworkManager, systemd-networkd) aplica el mismo patrón para colas de transmisión. En sistemas en la nube, el SPOOLING se generaliza con sistemas de colas de mensajes (RabbitMQ, Apache Kafka, AWS SQS).

5. Ejemplos de Manejadores de Dispositivos

Se presentan dos ejemplos representativos: el manejador del reloj (dispositivo sin transferencia de datos pero esencial para el SO) y el manejador de disco RAM (dispositivo más simple del tipo bloque).

5.1 Manejador del Reloj (Clock Driver)

El reloj (temporizador hardware) genera una interrupción periódica cada cierto intervalo fijo (tick). Este intervalo puede ser configurado por software. El manejador del reloj es invocado en cada tick y gestiona todas las funciones temporales del sistema:

Función	Descripción
Mantenimiento de la hora	Actualiza el contador de tiempo real del sistema (realtime clock).
Control del quantum	Decrementa el contador del quantum del proceso en ejecución; si llega a 0, invoca al planificador.
Alarmas de procesos	Gestiona una tabla de alarmas; al vencer una, envía SIGALRM al proceso correspondiente.
Watchdog timers	Ejecuta un callback previamente registrado al vencer un timeout (usado en drivers y kernel).
Estadísticas del sistema	Actualiza contadores de uso de CPU por proceso (utilizados por el planificador para calcular prioridades).

```
/* Esquema simplificado del manejador de interrupción del reloj */
void do_clocktick() {
    realtime++;
    /* actualizar hora del sistema */
}
```

```

/* Verificar alarmas pendientes */
if (next_alarm <= realtime) {
    buscar_proceso(&pp);
    cause_sig(pp, SIGALRM);      /* enviar señal al proceso */
    find_next_alarm(&next_alarm);
}

/* Gestionar quantum del proceso actual */
sched_ticks--;
if (sched_ticks == 0) {
    sched_ticks = SCHED_RATE;    /* recargar contador */
    reschedule();                /* invocar planificador */
}
}

```

🔴 **Nota actualizada:** En Linux, el tick del sistema tiene una frecuencia configurable (CONFIG_HZ: 100, 250 o 1000 Hz típicamente). Desde Linux 2.6.21 existe el tickless kernel (NO_HZ): el sistema puede suprimir los ticks cuando la CPU está ociosa, ahorrando energía. Los temporizadores de alta resolución (hrtimers) usan hardware de nanosegundos (TSC, HPET) independientemente del tick. En sistemas embebidos de tiempo real se usan timers con resolución de microsegundos.

5.2 Manejador de Disco RAM (RAM Disk Driver)

El disco RAM es un área de memoria principal reasignada para simular un dispositivo de bloque. Ofrece acceso instantáneo (latencia ~0) y es ideal para almacenamiento temporal de alta velocidad. La operación fundamental es copiar bloques entre el espacio de usuario y el área RAM del disco.

```

/* Esquema simplificado del driver de disco RAM */
#define ORIGEN  0x100000    /* dirección inicio del área RAM */
#define LIMITE  0x200000    /* dirección fin */

int do_mem(message *m_ptr) {
    if (m_ptr->posicion < 0) return ERROR;

    long mem_phys = ORIGEN + m_ptr->posicion;
    if (mem_phys > LIMITE) return 0;

    int count = m_ptr->count;
    if (mem_phys + count > LIMITE)      /* ajustar si excede límite */
        count = LIMITE - mem_phys;

    if (m_ptr->m_tipo == DISK_READ)
        memcpy(user_addr, (void*)mem_phys, count);    /* RAM → usuario */
    else
        memcpy((void*)mem_phys, user_addr, count);    /* usuario → RAM */

    return count;    /* bytes transferidos */
}

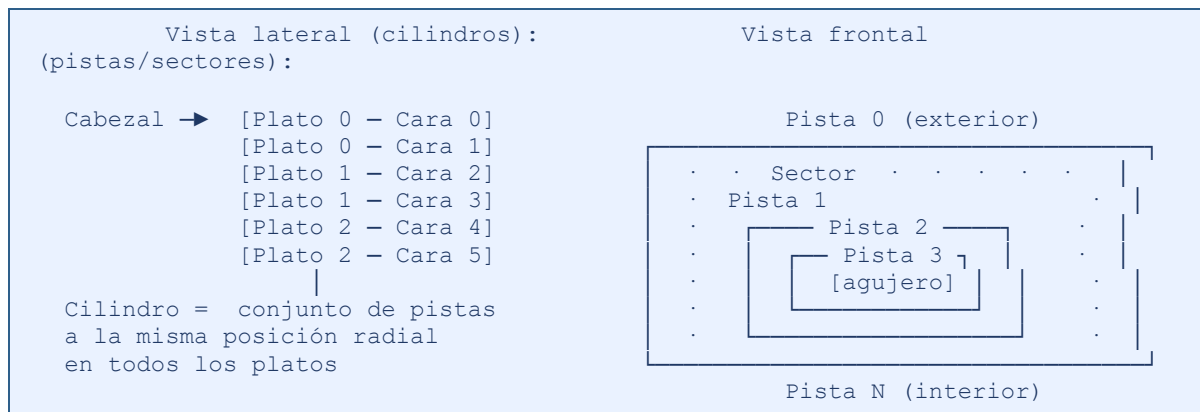
```

🔴 **Nota actualizada:** En Linux moderno, los discos RAM se crean con /dev/ram* (driver brd) o usando tmpfs/ramfs montados en memoria. Una alternativa más eficiente es zram: un disco RAM comprimido en tiempo real, usado en Android y dispositivos con poca RAM como swap comprimido. En sistemas de alta disponibilidad, los discos RAM se usan para directorios temporales (/tmp, /run) mejorando el rendimiento.

6. Discos Magnéticos: Características Físicas

Los discos magnéticos son el dispositivo de almacenamiento secundario más usado históricamente. Su comprensión es fundamental para entender los algoritmos de planificación de E/S.

6.1 Estructura Física del Disco



Elemento	Descripción y función
Plato (Platter)	Disco circular de aluminio o vidrio recubierto de material magnético. Un HDD típico tiene 2–8 platos.
Cara (Surface)	Cada plato tiene 2 caras magnetizadas. Un HDD con 4 platos tiene 8 superficies activas.
Pista (Track)	Anillo concéntrico en una cara. La dirección de pista es el segundo componente de la dirección en disco.
Cilindro (Cylinder)	Conjunto de pistas a la misma posición radial en todos los platos/caras. Es equivalente en dirección a la pista.
Sector	División de una pista. Unidad mínima de transferencia física. Tamaño tradicional: 512 bytes. Modern Advanced Format: 4096 bytes (4K).
Cabezal (Head)	Uno por superficie; montados en un brazo común que se mueve radialmente. Todos los cabezales se desplazan al unísono.

6.2 Tiempos de Acceso al Disco

Componente	Descripción	Valores típicos HDD
Tiempo de búsqueda (Seek time)	Tiempo en mover el brazo hasta la pista deseada. Es el componente más costoso y el objetivo de la planificación.	3–15 ms (promedio ~8 ms)
Tiempo latente (Rotational latency)	Espera a que el sector deseado llegue bajo el cabezal. Depende de las RPM del disco.	3–8 ms (7200 RPM → 4,2 ms prom.)
Tiempo de transferencia (Transfer time)	Tiempo en leer/escribir el sector mientras pasa bajo el	0,1–0,5 ms por sector

Componente	Descripción	Valores típicos HDD
	cabezal. Depende del tamaño del sector y las RPM.	
Tiempo total de acceso	Seek + Latency + Transfer. Para minimizar el rendimiento global se optimiza el seek time mediante planificación.	~10–25 ms (HDD) vs. ~0,1 ms (SSD)

💡 **Concepto clave:** Solo el tiempo de búsqueda puede optimizarse mediante algoritmos de planificación (decidiendo el orden en que se atienden las solicitudes). El tiempo latente depende de las RPM y no es controlable por el SO. El tiempo de transferencia es fijo para un sector dado. Por esto, todos los algoritmos de planificación de disco buscan minimizar el movimiento total del brazo.

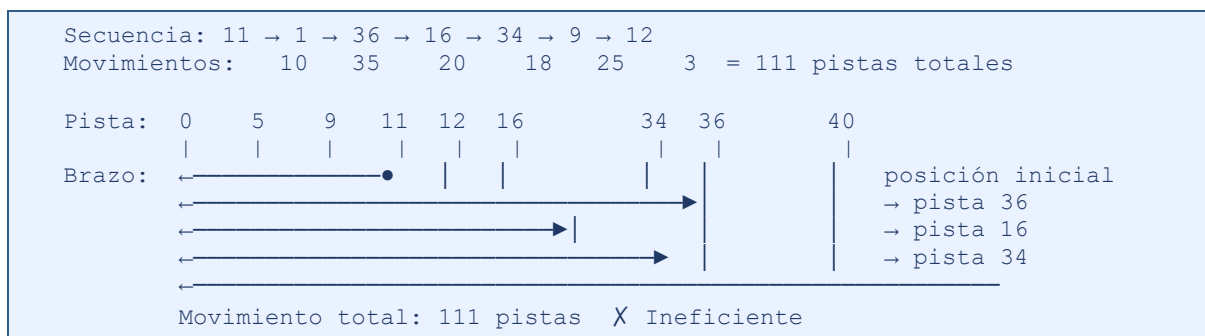
7. Algoritmos de Planificación de Disco

En un entorno multiprogramado existen colas de solicitudes de E/S pendientes para cada dispositivo de bloques. El planificador de disco decide el orden en que se atienden para minimizar el tiempo de búsqueda total. Se usará el siguiente ejemplo para ilustrar todos los algoritmos:

Ejemplo de referencia: Solicitudes pendientes en pistas: 1, 36, 16, 34, 9, 12. Posición actual del cabezal: pista 11. Total de pistas: 0-40.

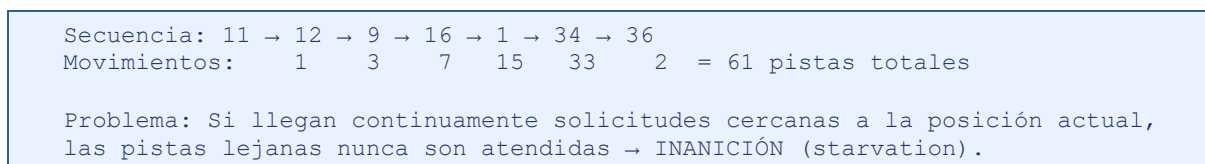
7.1 FCFS — First Come, First Served

Atiende las solicitudes en orden de llegada. Justo pero ineficiente, ya que puede provocar movimientos excesivos del brazo.



7.2 SSTF / SSF — Shortest Seek Time First

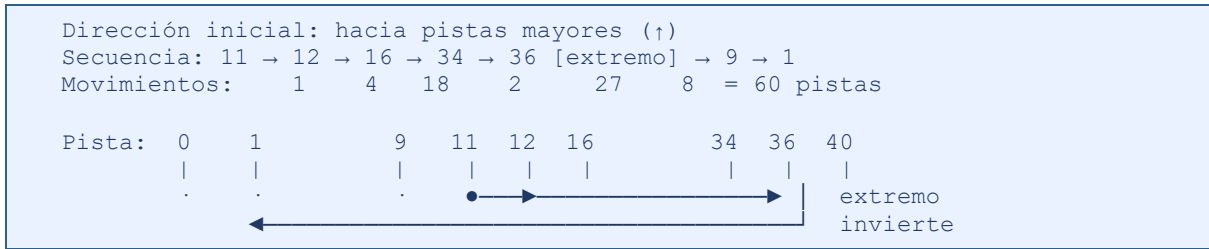
Atiende siempre la solicitud más cercana a la posición actual del cabezal. Muy eficiente en promedio, pero puede causar inanición (starvation) en solicitudes alejadas.



Algoritmo	Ventaja	Desventaja
FCFS	Justo, sin inanición.	Ineficiente: muchos movimientos innecesarios.
SSTF	Eficiente: minimiza seek total.	Inanición posible en solicitudes lejanas.

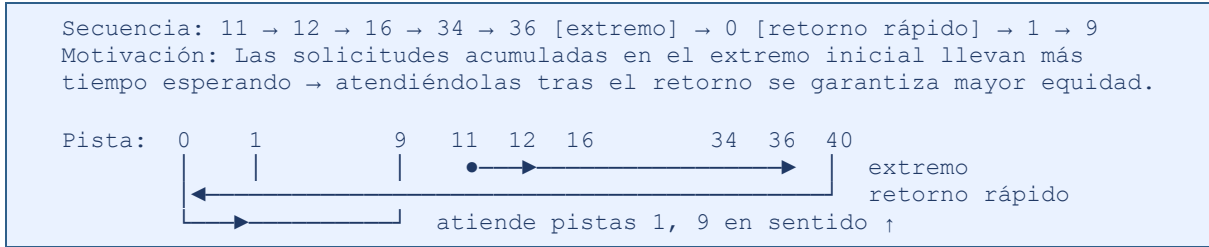
7.3 SCAN — El Algoritmo del Elevador

El brazo se mueve de un extremo al otro, atendiendo solicitudes en su camino. Al llegar al extremo, invierte el sentido. Similar al funcionamiento de un elevador. Combina eficiencia con equidad.



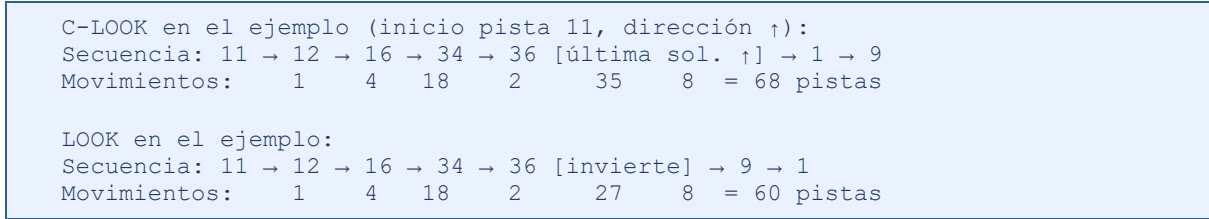
7.4 C-SCAN — Scan Circular

Variante del SCAN: el brazo siempre se mueve en el mismo sentido (ej.: de menor a mayor pista). Al llegar al extremo, regresa al inicio sin atender solicitudes en el retorno. Distribución más uniforme del tiempo de espera.



7.5 LOOK y C-LOOK — Variantes Prácticas

En la práctica, SCAN y C-SCAN no se llevan hasta el extremo físico del disco: el brazo solo llega hasta la última solicitud en esa dirección y luego invierte (LOOK) o retorna (C-LOOK). Así se evitan movimientos innecesarios hasta el extremo.



Algoritmo	Pistas totales	Inanición	Preemptive	Equidad
FCFS	111	No	No	Alta

Algoritmo	Pistas totales	Inanición	Preemptive	Equidad
SSTF	61	Posible	No	Media
SCAN	60	No	No	Alta
C-SCAN	~68	No	No	Muy alta
LOOK	60	No	No	Alta
C-LOOK	68	No	No	Muy alta

⚠ Importante: La elección del algoritmo depende del patrón de uso: SSTF para cargas con solicitudes cercanas y sin riesgo de starvation; LOOK/C-LOOK para cargas mixtas con buen balance de eficiencia y equidad. Para SSDs estos algoritmos son irrelevantes (no hay partes móviles) y se usan algoritmos basados en prioridad o FIFO simple.

8. Tecnologías Modernas de Almacenamiento

Las tecnologías de almacenamiento han evolucionado radicalmente desde los HDD tradicionales. El conocimiento de estas tecnologías es fundamental para comprender el diseño de los SO actuales.

8.1 SSD y NVMe

Aspecto	HDD	SSD/NVMe
Tecnología	Platos magnéticos + cabezales móviles	Memoria NAND Flash (sin partes móviles)
Tiempo de acceso	~10–15 ms (seek + latency)	~0,03–0,1 ms (SATA SSD) / ~0,01 ms (NVMe)
Ancho de banda	100–200 MB/s (HDD mecánico)	500 MB/s (SATA SSD) / 3–7 GB/s (NVMe PCIe 4.0)
Planificación E/S	SCAN/LOOK son óptimos	NOOP/NONE (sin reordenamiento) o mq-deadline
Desgaste	Sin límite de escrituras (mecánico)	Ciclos P/E limitados: 3000–100000 (TBW)
Uso óptimo	Almacenamiento masivo, backups, archivos fríos	SO, aplicaciones, bases de datos, acceso aleatorio

8.2 RAID — Redundant Array of Independent Disks

RAID combina múltiples discos físicos para mejorar el rendimiento, la capacidad o la redundancia. El SO (o hardware dedicado) presenta los discos físicos como uno lógico.

Nivel	Descripción	Ventaja	Desventaja
RAID 0	Striping: datos distribuidos entre N discos sin redundancia.	Máx. rendimiento ($N \times \text{velocidad}$).	Sin tolerancia a fallos.

Nivel	Descripción	Ventaja	Desventaja
RAID 1	Mirroring: datos duplicados en 2 discos.	Tolerancia total a fallo de 1 disco.	50% de capacidad efectiva.
RAID 5	Striping con paridad distribuida entre $N \geq 3$ discos.	Balance rendimiento/redundancia.	Escritura penalizada (cálculo paridad).
RAID 6	Como RAID 5 pero con doble paridad.	Tolera fallo de 2 discos simultáneos.	Mayor overhead de escritura.
RAID 10	RAID 1+0: Mirroring + Striping.	Alto rendimiento y redundancia.	Requiere mínimo 4 discos; coste alto.

🔴 **Nota actualizada:** En Linux, el RAID software se gestiona con mdadm (/dev/md*). El Device Mapper (dm-raid) es la implementación moderna. En la nube (AWS, GCP, Azure), el almacenamiento se abstrae completamente: los volúmenes EBS, los Persistent Disks o los Azure Managed Disks presentan una interfaz de bloque estándar independientemente de si el backend usa HDD, SSD, RAID o almacenamiento distribuido (como Ceph o similar).

8.3 Comparativa General de Tecnologías de Almacenamiento

Tecnología	Latencia	Ancho de banda	Planificación SO
HDD mecánico	10–15 ms	150–200 MB/s	SCAN / LOOK / C-LOOK
SATA SSD	0,05–0,1 ms	500–600 MB/s	NOOP / mq-deadline
NVMe (PCIe 4.0)	0,01–0,02 ms	3.000–7.000 MB/s	NONE (mínima latencia)
RAM / zRAM	< 0,001 ms	10.000+ MB/s	Sin planificación
Almacen. nube (S3)	50–200 ms	Variable (red)	Transparente al SO

9. Estudio Independiente

- ¿Por qué el sistema operativo interactúa con el controlador del dispositivo y no directamente con el dispositivo físico? ¿Qué ventajas aporta esta separación?
- Explique el mecanismo DMA paso a paso. ¿Qué es la coherencia de caché y por qué se convierte en un problema con DMA en sistemas con caché?
- Describa las cinco metas del software de E/S. ¿Cuál de ellas considera más difícil de lograr y por qué?
- ¿Cuál es la diferencia entre un manejador de interrupción (interrupt handler) y un manejador de dispositivo (device driver)? ¿Por qué en Linux se separa la ISR en Top Half y Bottom Half?
- Dado el conjunto de solicitudes de pistas: 98, 183, 37, 122, 14, 124, 65, 67 y posición inicial del cabezal en pista 53 (moviéndose hacia mayor), calcule el total de movimiento del brazo para: FCFS, SSTF, SCAN, C-SCAN, LOOK y C-LOOK. ¿Cuál es el más eficiente?
- ¿Por qué los algoritmos de planificación de disco (SCAN, SSTF, etc.) son irrelevantes para los SSD NVMe? ¿Qué algoritmo se usa en Linux para estos dispositivos y por qué?
- Explique el concepto de SPOOLING. ¿Cómo lo implementa Linux para la impresión? ¿En qué se parece a una cola de mensajes moderna (como Kafka o RabbitMQ)?

8. ¿Qué es RAID 5? ¿Cuántos discos como mínimo requiere? Si un disco falla, ¿cómo se reconstruye la información? ¿Por qué RAID no es un sustituto del backup?

— Fin de la Conferencia 5 —